Week 3: Data Wrangling

September 9, 2026

THE POINT OF THIS WEEK

A data frame is just a table: columns are variables (all one type each), rows are observations. Almost everything you'll do this semester — cleaning a survey export, building a choice dataset, summarizing results — is a sequence of small, composable operations on a data frame. The **tidyverse** package `dplyr` gives you five verbs that cover nearly all of it, and the pipe lets you chain them into a readable sentence instead of a nested mess.

> Every wrangling task is a chain of verbs joined by `%>%`. Read the pipe as the word "…and then…": `data %>% filter(...) %>% select(...)` means "take data, and then keep some rows, and then keep some columns."

Data frames: rows, columns, `$`

- **Columns** are vectors (`beatles$firstName`) — all values the same type.
- **Rows** are observations, sliced with `df[row, ]` (e.g. `beatles[1, ]`).
- `nrow()`, `ncol()`, `dim()` — size. `glimpse()` — every column's type at a glance. `head()` / `tail()` — first / last 6 rows.

Getting data into R

Step	Code
1. Path to the file	`path <- file.path('data', 'data.csv')`
2. Read it in	`data <- read_csv(path)`

Use **`read_csv()`**, not `read.csv()` — better defaults, and it returns a tibble. **Never double-click a `.csv` and open it in Excel** — Excel silently rewrites dates, IDs, and leading zeros. If you must look, open a *copy*.

THE DPLYR VERBS

Verb	What it does
`select()`	Keep/drop columns by name
`filter()`	Keep rows matching a condition
`arrange()`	Sort rows by column(s)
`mutate()`	Create new columns
`summarize()`	Collapse to summary values

`select()` — columns

```
beatles %>% select(firstName, lastName)   # keep these
beatles %>% select(-firstName)            # drop this
beatles %>% select(ends_with("Name"))     # by pattern
```

Helper	Matches
`ends_with()`	Column names ending in a string
`contains()`	Column names containing a string
`matches()`	Column names matching a regex
`one_of()`	Names from a given vector

`filter()` — rows

```
beatles %>% filter(yearOfBirth > 1941, deceased == FALSE)
```

Multiple comma-separated conditions are combined with **AND**.

Meaning	Operator
Greater / less than	`>` / `<`
Greater-or-equal / less-or-equal	`>=` / `<=`
Equal to	`==`
Not equal to	`!=`
In a set	`%in% c(1, 4)`
Not missing	`!is.na(x)`

`mutate()` — new columns

```
beatles %>% mutate(age = 2022 - yearOfBirth)
beatles %>% mutate(playsGuitar = ifelse(instrument == "guitar", 1, 0))
```

A column created earlier in the same `mutate()` can be used immediately by a later one, in the same call.

`arrange()` — sorting

```
beatles %>% arrange(yearOfBirth)          # ascending
beatles %>% arrange(desc(yearOfBirth))    # descending
```

COMMON WRANGLING MISTAKES

Symptom	Cause
`filter()` keeps everything / errors	Used `=` instead of `==`
Filter on text finds nothing	Forgot quotes: `state == DC` vs `state == "DC"`
Changes don't stick	Forgot to reassign: needs `data <- data %>% ...`
Dates/IDs look wrong after import	Opened the `.csv` in Excel first
Weird column types after import	Used `read.csv()` instead of `read_csv()`

PROJECT PROPOSALS

Every product analysis starts the same way: know your customer, know your competitors, and know what design trade-offs are actually worth studying.

Proposal item	Description
Abstract	Product / technology in a few sentences
Introduction	Description, picture, background
Market Opportunity	Customer, competitors, market size
Product Attributes	2–4 key design/performance variables
Research Questions	2–4 questions the project should answer
Open Questions	What's still unresolved

This week's focus: identify your customer and competitors, then start drafting **Product Attributes** (features your customer cares about) and **Research Questions** (the decisions those attributes should inform). Start with a rough bullet list and more items than you need — cutting later is easier than starting from nothing.
Full guidelines: project/1-proposal.html.

BEFORE NEXT CLASS

- **No lecture next week** — Sep 16 is Team Proposal Workshops: 1-on-1 team meetings with Prof. Helveston instead of class. Sign up for a slot.
- **Project Proposal due Sep 20** — build out the model relationships table (example sheet) with your team's customer, competitors, product attributes, and research questions.
- HW3 (Quarto & Plotting) is due **Sep 21**, ahead of the Sep 23 class.

You should be able to:

- Import a `.csv` with `file.path()` + `read_csv()` without hard-coding a path.
- Chain `select()`, `filter()`, `mutate()`, and `arrange()` with `%>%` to answer a question about a dataset.
- Explain the difference between `=` and `==`, and why you must reassign a pipeline's output to keep it.
- Describe your project's likely customer, competitors, and 2–4 candidate product attributes.